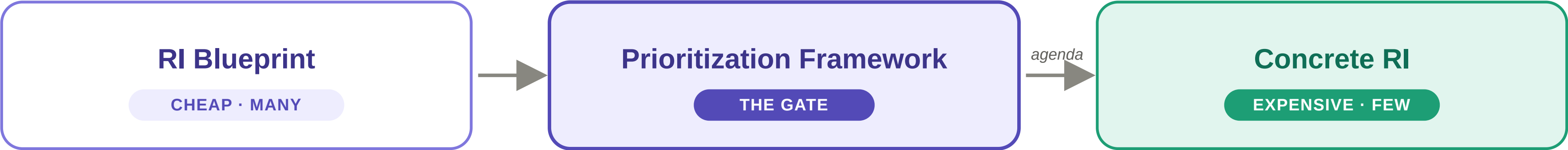


PART 3

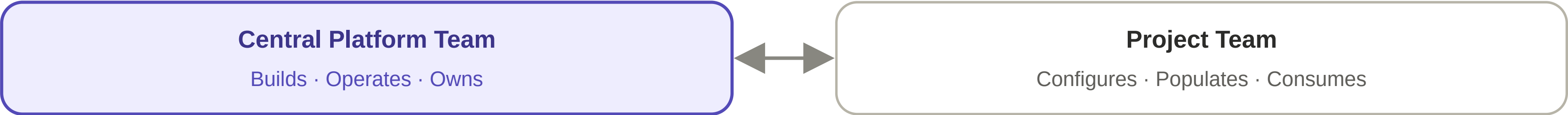
THE PROCESS

Part 3 — The Process

Turning a candidate standard into something a team can actually build.



Then: who builds it, and who owns it?



The gate guarantees a control exists; the project owns its quality; governance owns the consequences.

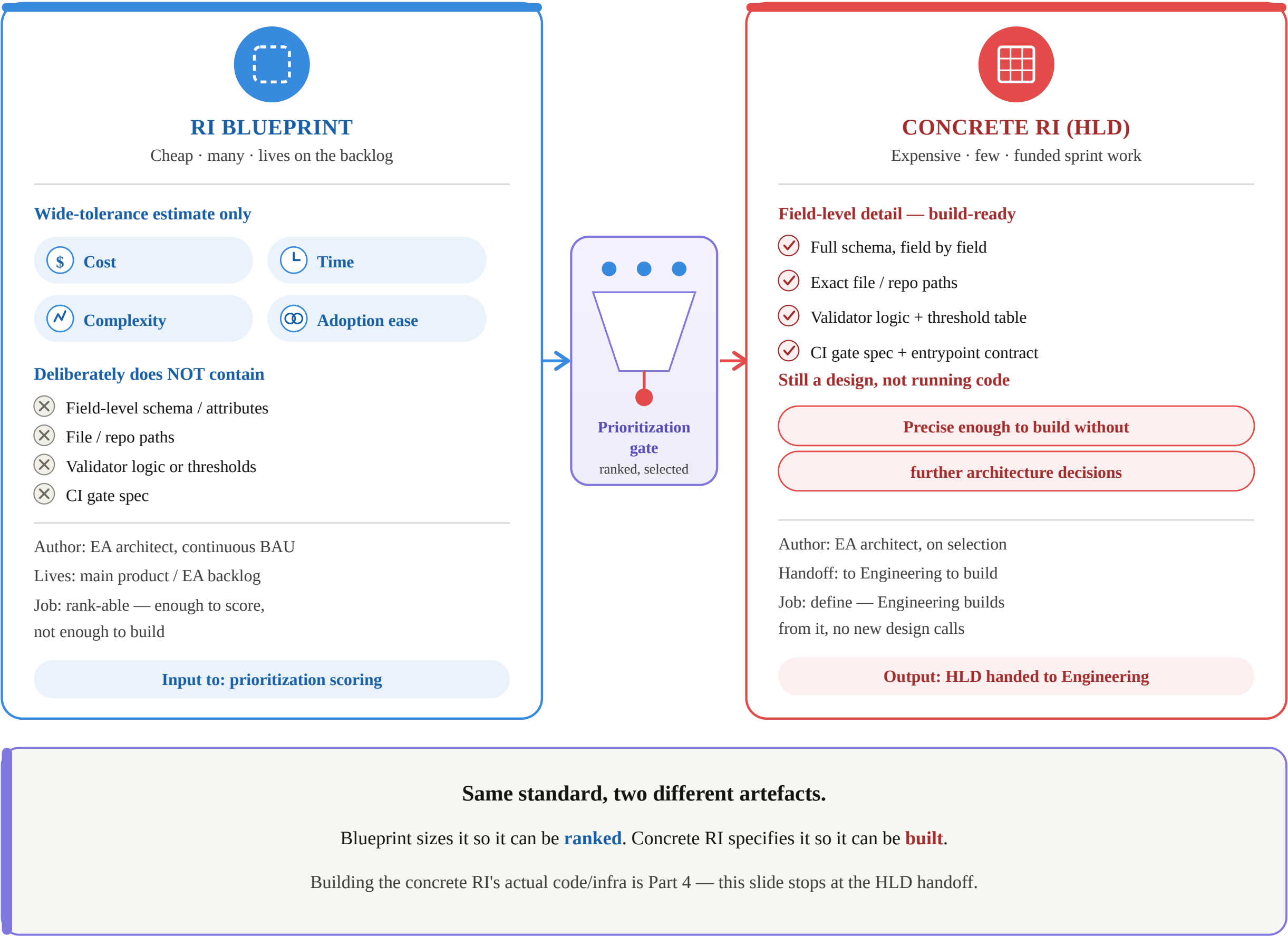
We'll also touch on the operating-model org chart

Head of AI · Lead Architect · Head of Engineering · Head of Product · Head of Governance

How they all connect — that reveal is live in the session.

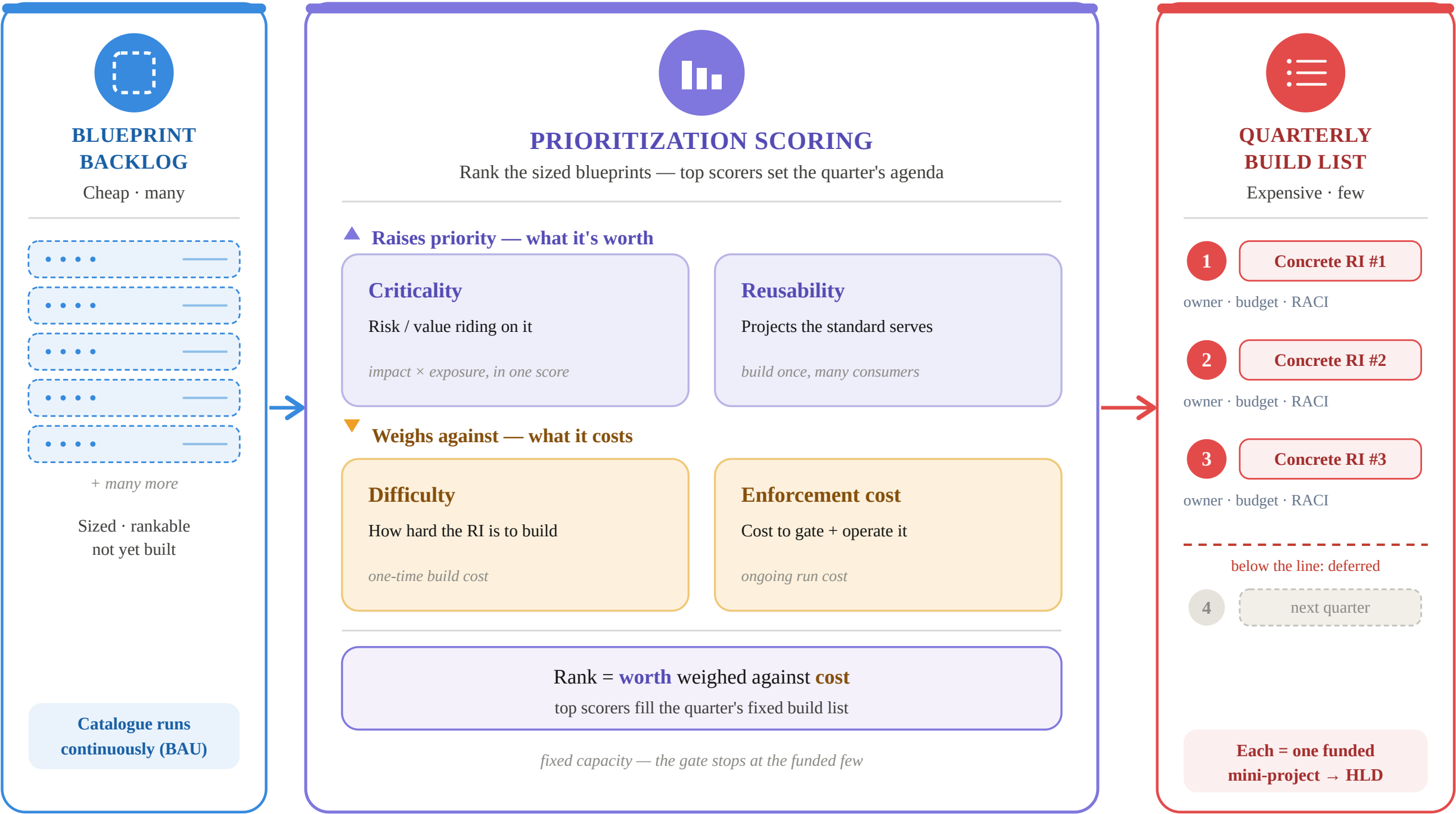
RI Blueprint vs. Concrete RI

Two different artefacts, on either side of the prioritization gate



The Prioritization Gate — Scoring the Blueprints

Four dimensions rank the sized backlog into the quarter's fixed build list



Prioritization is the gate — cheap-many in, funded-few out.

Rank each sized blueprint by its **worth** (**criticality** · **reusability**) against its **cost** (**difficulty** · **enforcement**).

The quarter's winners each become one funded, owned mini-project — a concrete RI (HLD) for Engineering.

Worked Example — Prioritizing Two Standards

Two real catalogue standards, scored on the four dimensions over the next three slides

ST-GS1B3-01

P2 · Securityscalingimpact: High

STANDARD

Never let a user retrieve what they aren't allowed to see

ASPIRATION

"No one should learn through an AI system what they couldn't learn directly."

WHAT IT IS

Per-query, identity-scoped retrieval from shared vector stores — a central SDK applies an entitlement pre-filter so a user only gets chunks their roles permit.

PROFILE (hypothesis)

Legal/security-critical · expensive to build

high worth, high cost — the score has to confirm it

VS

ST-GC3B3-01

P5 · Cost Optimizationscalingimpact: Medium

STANDARD

Don't pay twice for the same prompt content

ASPIRATION

"The same work is never paid for twice."

WHAT IT IS

Every prompt template declares its static-prefix token count and a cache decision; a CI gate fails the build when an eligible prefix is left uncached or the count is faked.

PROFILE (hypothesis)

Cost-only · cheap to build

low worth, low cost — a cheap win, but not urgent

SCORED ON THE FOUR DIMENSIONS

▲

RAISES PRIORITY — worth

Criticality

Reusability

▼

WEIGHS AGAINST — cost

Difficulty

Enforcement cost

Scoring GS1B3-01 — Permission-Aware Retrieval

Example 1 of 2 · the use case, and what it costs us to NOT have it

ST-GS1B3-01

P2 · Security

scaling

impact: High

Never let a user retrieve what they aren't allowed to see

"No one should learn through an AI system what they couldn't learn directly."

WHAT IT MEANS

A RAG app retrieves from a shared vector store that indexes many tenants and teams into one index. This standard scopes every retrieval to the caller: documents are ACL-labelled at ingestion, and a central SDK applies an identity pre-filter.

THE FAILURE IT PREVENTS

Embedding strips a document's source permissions — a confidential contract and a public FAQ become vectors that look identical. An unfiltered top-k then hands the user (and the model answers from) chunks they may never read.

WHY IAM & CLASSIFICATION MISS IT

- IAM gates whether the **app** can reach the store — not which chunks within it a given user's query may surface.
- Model-layer classification is too late: the leak is at retrieval, before the model ever reasons over the text.

CONCRETE EXAMPLE

A support agent's query must never surface another tenant's contract — even though its vector sits right beside the public FAQ they may see.

SCORED ON THE FOUR DIMENSIONS

▲ Criticality

● ● ● ● ● 5 / 5

Cross-tenant retrieval of PII / confidential data is direct legal exposure — GDPR, OWASP broken access control. The catalogue's first legal-exposure veto.

evidence: RI tier mandatory_centralise (D1 legal_exposure = yes) · impact High

▲ Reusability

● ● ● ● ● 5 / 5

Every workload retrieving from a shared / multi-tenant vector store. One retrieval SDK + acl schema serves all RAG apps — the failure recurs everywhere.

evidence: central retrieve() SDK + pinecone / opensearch / pgvector / bedrock adapters

▼ Difficulty

● ● ● ● ○ 4 / 5

Identity pre-filter inside the ANN search (not a post-filter), IdP entitlement resolution, per-backend adapters, acl labelling, 3 CI lints — weeks of work.

evidence: RI D2 repeatability_cost = 3 (high) · rebuilt locally = weeks per project

▼ Enforcement cost

● ● ● ● ○ 4 / 5

The lints are mechanical, but the residual label-correctness gap needs a security-owned quarterly audit + central review at the gate. Not fire-and-forget.

evidence: audit_mode central_review_at_gate · validator enterprise_security

Worth ▲ 10 / 10 (crit 5 + reuse 5) Cost ▼ 8 / 10 (diff 4 + enf 4)

Verdict: BUILD FIRST — fund it fully this quarter.

The criticality-5 legal / security veto makes it mandatory. High difficulty and enforcement cost do NOT demote it — you absorb the cost and pull its soft-dependency observability (GO3B2-01) in alongside.

Scoring GC3B3-01 — Prompt Caching

Example 2 of 2 · same four dimensions, a very different profile

ST-GC3B3-01 · Don't pay twice for the same prompt content

"The same work is never paid for twice." · P5 Cost Optimization · scaling · impact Medium

Criticality ▲

● ● ○ ○ ○ 2 / 5

No legal or regulatory exposure — pure cost discipline. The only harm is a big fixed prefix re-billed at full rate, showing on the inference bill ~30 days late. The output stays correct.

EVIDENCE

RI D1 legal_exposure = no · impact_level Medium · u_value: cost, not customer or regulator

Reusability ▲

● ● ● ○ ○ 3 / 5

Applies to any workload with a large, stable prompt prefix (system instructions, fixed schema, static context) — but only pays off where the prefix is big and reused often enough to beat cache write cost.

EVIDENCE

extends GO3B1-01 registry · eligibility keyed to per-model cache minimums (cache_minimums.yaml)

Difficulty ▼

● ● ○ ○ ○ 2 / 5

Two lints (cache-eligibility + recompute-variance) bolted onto the existing llm-sdk suite; reuses GO3B1-01's tokenizers and manifest. Thin build — no new spine to stand up.

EVIDENCE · SEQUENCING

RI D2 = 2 · but HARD-depends on GO3B1-01 (prompt registry) — that prereq must be built first

Enforcement cost ▼

● ● ○ ○ ○ 2 / 5

Fully mechanical: self-attestation with the CI logs as evidence, no central human review at the gate. Ongoing cost is quarterly cache_minimums.yaml upkeep + a shared tokenizer-accuracy review.

EVIDENCE

audit_mode: self_attestation_with_mechanical_evidence · validator: project_architect

Worth ▲ 5 / 10 (criticality 2 + reusability 3)

Cost ▼ 4 / 10 (difficulty 2 + enforcement 2)

Verdict: FAST FOLLOW — a cheap win, but it waits.

Criticality 2 means it can't jump the queue. Its low cost and fair reuse make it easy to slot in after the critical builds — and only once its GO3B1-01 prerequisite (the prompt registry) is in place.

Prioritization Verdict — GS1B3-01 vs GC3B3-01

Criticality gates the queue; cost only sizes it · each dimension scored 1–5

DIMENSION	ST-GS1B3-01 retrieval auth	ST-GC3B3-01 prompt caching
▲ Criticality	5 /5	2 /5
▲ Reusability	5 /5	3 /5
WORTH ▲ (pull)	10 /10	5 /10
▼ Difficulty	4 /5	2 /5
▼ Enforcement cost	4 /5	2 /5
COST ▼ (drag)	8 /10	4 /10

GS1B3-01 → BUILD FIRST

this quarter, funded fully

Criticality 5 is a legal/security veto — it is mandatory. Cost 8 is high, but cost never demotes a critical item; you absorb it and pull its hard prerequisites in alongside.

GC3B3-01 → FAST FOLLOW

fills leftover capacity

Criticality 2 keeps it out of the top slot. But cost 4 and fair reuse make it a cheap win — slot it in after the critical builds, once its GO3B1-01 prerequisite exists.

The lesson: the two costs never bought GC3B3-01 ahead of GS1B3-01.

Criticality gates the queue — it decides what MUST be built. A legal/security veto is non-negotiable.

Cost only sizes it — difficulty + enforcement decide how much of the quarter each winner eats, not whether it wins.

Reusability breaks ties and justifies central build; a cheap, easy standard fills the gaps around the critical ones.

A cheap, easy standard never outranks a legally-critical one — it just waits its turn.